

PeerLink / P2P Internal Cybersecurity Remediation Report

Comprehensive internal remediation notes for the 11 KACST cybersecurity findings and related application follow-up.

Purpose

This internal report documents the 11 KACST cybersecurity findings for PeerLink / P2P, the reason each item mattered, what was required to resolve it, what was changed, and how the fixes were verified.

This report is intended for internal engineering and operations tracking. It is more detailed than the external/team-facing summary report.

Overall Status

All 11 reported findings have been addressed. The remediation covered infrastructure hardening, HTTPS enforcement, upload validation, dependency cleanup, security headers, MFA/OTP behavior, input validation, HTTP/2 enablement, and corporate email-change restrictions.

For internal tracking, the fixes were split between items I handled directly and items handled by Umair through application changes. I handled the infrastructure/security-header/deployment items and review/merge follow-up. Umair handled the main authentication, email-domain, input-validation, and draft-related application fixes.

1. Outdated NGINX 1.24.0 with Critical/High Vulnerabilities

Owner: Hamza/Nemo — infrastructure hardening on OCI.

Finding

The public web server exposed `nginx/1.24.0 (Ubuntu)`, which triggered the scanner finding that the NGINX version was outdated and affected by critical/high vulnerabilities.

Why It Mattered

Even if Ubuntu has backported security patches, exposing the exact upstream version creates a scanner target and allows attackers to fingerprint the stack easily.

What Was Needed

- Confirm the actual installed NGINX package on the OCI host.
- Confirm whether Ubuntu security repositories had a newer patched package.
- Avoid risky production package-source changes unless necessary.
- Hide the public NGINX version banner.
- Reload NGINX safely and verify headers externally.

Remediation Performed

The host package was checked on the OCI instance. The installed version was the latest patched Ubuntu Noble package available from the configured repositories. The NGINX public version banner was then hidden using `server_tokens off;`.

Verification

Public response headers no longer show the exact NGINX/Ubuntu version. The server now returns only a generic NGINX server header.

Notes

This finding was closely related to issue 6, web server version disclosure. The same hardening addressed both the scanner trigger and the exposed-version concern.

2. Login Exposed Over Plain HTTP

Owner: Hamza/Nemo — HTTPS and NGINX production configuration.

Finding

Login and authentication pages were reachable over plain HTTP.

Why It Mattered

Plain HTTP can expose credentials, session-related traffic, and authentication flows to interception or downgrade risks.

What Was Needed

- Enable HTTPS for `p2p.iiootsolutions.sa`.
- Issue and install a valid TLS certificate.
- Configure NGINX to listen on HTTPS.
- Redirect HTTP traffic to HTTPS.
- Ensure application domain settings used HTTPS.
- Verify login and application pages still worked after the change.

Remediation Performed

Let's Encrypt HTTPS was enabled on the OCI NGINX reverse proxy. HTTP traffic was redirected to HTTPS. Application domain configuration was aligned to use the HTTPS public domain.

Verification

The site loads over HTTPS, HTTP routes redirect to HTTPS, and the authenticated application flow was smoke-tested successfully.

Notes

This fix also prepared the environment for HSTS and HTTP/2.

3. Unrestricted File Upload Vulnerability

Owner: Hamza/Nemo — upload validation hardening and deployment verification.

Finding

Upload flows did not have enough protection against unsafe, spoofed, or unauthorized uploads.

Why It Mattered

Unrestricted upload handling can allow malicious files, content-type spoofing, oversized payloads, extension mismatches, and uploads to objects a user does not own.

What Was Needed

- Review all user-facing media upload paths.
- Enforce allowed file types.
- Validate file content rather than trusting browser-provided content type.
- Block extension/content mismatches.
- Apply size checks.
- Ensure uploaded files use safe extensions.
- Enforce ownership checks before allowing uploads.
- Test invalid and valid upload cases.

Remediation Performed

Upload validation was hardened. The implementation validates file type, file content, extension consistency, file size, and ownership before accepting uploads. Unsafe renamed files and unauthorized uploads are blocked.

Verification

Test cases confirmed fake image uploads, mismatched extensions, mismatched MIME declarations, and unauthorized media uploads are rejected.

Notes

Future optional hardening could include antivirus scanning and stricter object-storage serving headers if required by policy.

4. Login Implemented Without 2FA/MFA

Owner: Umair — OTP/MFA application changes; Hamza/Nemo reviewed, merged, and verified.

Finding

The login flow did not enforce a second factor.

Why It Mattered

Password-only login increases risk if credentials are leaked, reused, guessed, or phished.

What Was Needed

- Add OTP-based verification behavior for login.
- Support OTP challenge handling and resend behavior.
- Ensure successful verification creates the correct session state.
- Avoid confusing success messages when session creation fails.
- Test login and verification flows in production.

Remediation Performed

OTP-based verification/MFA behavior was added to the authentication flow. Challenge handling, resend behavior, session creation handling, and post-verification routing were also corrected.

Verification

A live production login smoke test confirmed the OTP flow works and the application can be accessed after verification.

Notes

Trusted-device behavior is present so the experience can remain usable while still improving login security.

5. Outdated and Vulnerable Frontend Libraries

Owner: Hamza/Nemo — dependency audit/update and deployment verification.

Finding

The frontend dependency tree contained vulnerable packages.

Why It Mattered

Frontend dependency vulnerabilities can expose the application to known client-side or build-chain security issues.

What Was Needed

- Run frontend dependency audit.
- Update patched package versions through the lockfile.
- Rebuild the frontend.
- Confirm audit results after the update.
- Deploy and verify the live frontend still loads correctly.

Remediation Performed

The frontend dependency lockfile was updated, the build was tested, and the updated frontend was deployed.

Verification

The audit result was reduced to 0 vulnerabilities, and the live frontend loaded successfully after deployment.

Notes

This should be repeated periodically because frontend dependency findings can reappear as packages age.

6. Web Server Version Disclosure via HTTP Response Headers

Owner: Hamza/Nemo — NGINX header hardening.

Finding

The web server exposed its exact NGINX and Ubuntu version in HTTP response headers.

Why It Mattered

Version disclosure helps attackers and scanners identify the exact stack and map it to known vulnerabilities.

What Was Needed

- Remove exact version disclosure from public headers.
- Keep the production reverse proxy stable.
- Verify the public header no longer contains version details.

Remediation Performed

NGINX `server_tokens off;` was enabled on the host reverse proxy.

Verification

Public headers no longer expose `nginx/1.24.0 (Ubuntu)`.

Notes

This was handled together with issue 1.

7. Missing HTTP Strict Transport Security / HSTS Header

Owner: Hamza/Nemo — NGINX security header configuration.

Finding

The application did not return an HSTS header.

Why It Mattered

Without HSTS, browsers may still attempt insecure HTTP connections, especially on first access or after user/browser behavior that downgrades requests.

What Was Needed

- First ensure HTTPS is stable.
- Add HSTS to HTTPS responses.
- Verify the header appears on public responses.

Remediation Performed

HSTS was added after HTTPS was enabled and verified.

Verification

HTTPS responses include `Strict-Transport-Security`.

Notes

HSTS should only be enabled after HTTPS is confirmed stable, which was done.

8. Missing Content Security Policy / CSP Header

Owner: Hamza/Nemo — NGINX CSP configuration and browser verification.

Finding

The application did not return a Content Security Policy header.

Why It Mattered

CSP reduces the impact of browser-side injection attacks by limiting where scripts, styles, images, and other resources can load from.

What Was Needed

- Inspect what the frontend needs to load.
- Add a CSP that improves security without breaking the app.
- Allow required trusted sources only.
- Test core pages after the header is applied.

Remediation Performed

A safe CSP header was added at the NGINX layer. It was designed to reduce injection risk while still allowing required frontend resources.

Verification

The CSP header appears on public responses, and BrowserOps checks confirmed the application still loads after the header was applied.

Notes

CSP can be tightened further over time if the frontend removes inline styles or reduces third-party resource needs.

9. Lack of Input Validation Across Application Fields

Owner: Umair — backend/application validation changes; Hamza/Nemo reviewed, merged, and verified.

Finding

Application forms accepted suspicious input, invalid values, and scanner-style payloads.

Why It Mattered

Weak input validation can allow stored junk data, injection attempts, XSS payloads, malformed business data, and scanner payloads to persist in the system.

What Was Needed

- Identify exposed forms and user-input fields.
- Add server-side validation, not only frontend validation.
- Reject common dangerous patterns.
- Enforce length limits and valid field ranges.
- Validate list sizes, tags, categories, and nested form data.
- Test valid and invalid submissions.

Remediation Performed

Input validation was added for major user-submitted application fields. The application now rejects suspicious payloads, invalid values, excessive lengths, invalid categories, unsafe tags, and common scanner strings.

Verification

Validation tests were run and passed. Manual checks confirmed invalid payloads are rejected while valid data remains accepted.

Notes

This should remain an ongoing area of review as new forms and fields are added.

10. Application Implemented Using HTTP/1.1

Owner: Hamza/Nemo — HTTP/2 enablement on production HTTPS endpoint.

Finding

The public endpoint was reported as using HTTP/1.1.

Why It Mattered

HTTP/2 is the expected modern protocol for secure web delivery and is commonly flagged when missing.

What Was Needed

- Enable HTTP/2 on the HTTPS listener.
- Verify HTTPS still works correctly.
- Confirm the public endpoint responds over HTTP/2.

Remediation Performed

HTTP/2 was enabled on the secure public NGINX endpoint.

Verification

The public HTTPS endpoint responds successfully using HTTP/2.

Notes

HTTP/2 enablement depended on the HTTPS work in issue 2.

11. Unauthorized Change of Corporate Email to Personal Email After Login

Owner: Umair — email-domain restriction changes; Hamza/Nemo reviewed, merged, and verified.

Finding

A user could change their account email from a corporate email to a personal email after login.

Why It Mattered

Corporate-domain enforcement is important for identity, access control, accountability, and organization-based user trust.

What Was Needed

- Block personal email domains during email-change operations.
- Enforce organization-domain matching.
- Validate before applying account updates.
- Add user-facing validation so invalid email changes are rejected clearly.
- Verify that invalid password/email-change cases do not trigger broken session behavior.

Remediation Performed

Email-change rules were strengthened. Users can no longer switch from an approved corporate email to a personal email, and organization-domain matching is enforced.

Verification

Personal-domain email changes and mismatched organization-domain changes are blocked.

Notes

The allowed-domain logic should be reviewed if the business later supports multiple corporate domains or partner organizations.

Additional Issue: Use Case Draft Save Failure

Owner: Umair — draft-save/application fix; Hamza/Nemo reviewed, merged, and applied follow-up verification.

Finding

After the main cybersecurity fixes, there was an additional application issue where use-case drafts could fail to save correctly, especially when partial draft data did not satisfy the stricter final-submit validation rules.

Why It Mattered

Drafts are supposed to allow incomplete work to be saved. If draft saves use the same strict validation as final submission, users can lose progress or be blocked while still preparing a use case.

What Was Needed

- Separate draft-save behavior from final-submit behavior.
- Allow partial draft data where appropriate.
- Keep strict validation for final submission.
- Verify that draft saves no longer fail unexpectedly.

Remediation Performed

The draft-save flow was corrected so partial draft data can be saved while final submission remains stricter. Follow-up validation confirmed the draft behavior works without weakening final submission requirements.

Verification

The draft-save issue was reviewed after the fix, merged, deployed, and verified through follow-up testing.

Final Internal Summary

The remediation work addressed all 11 KACST findings, plus the additional use-case draft-save issue. The most important changes were HTTPS enforcement, server header hardening, upload restrictions, dependency updates, CSP/HSTS headers, MFA/OTP behavior, input validation, HTTP/2, corporate email-change restrictions, and corrected draft-save behavior.

Recommended Follow-Up

1. 1. Re-run a security scan after the next production deployment.
2. 2. Keep frontend dependency audits in the release checklist.
3. 3. Add upload validation tests and input validation tests to CI if not already enforced.
4. 4. Periodically review CSP and tighten it where possible.

5. 5. Review new forms for validation before release.
6. 6. Keep production environment variables explicit and avoid development-style logging in production.
7. 7. Keep this report updated if KACST asks for evidence or retesting notes.